CATCH THE ROCKET

Real-Time Rocket Flight Analysis on ESP32



Team : ComputeX
Sourabha B N
C Pranusha
Nayeem Ahmed

WHAT ARE WE SOLVING?

Real-Time Flight Parameter Extraction from 100k Tracker Points

Flight Parameters to Find

- Gravity estimation (*g*)
- Apogee time & peak altitude
- Parachute deploy time & altitude
- Terminal descent velocity
- Touchdown / landing time
- Glitch rejection & arbitrary ALT <t> queries

System Constraints

- Strict RAM Limit: ⩽ 4,096 bytes (4 KB)
- Strict Latency: ⩽ 12,000 CPU cycles per sample
- Streaming Tracker: Exactly 100,000 calls
- Generalization: Must work on unseen flights without hardcoding





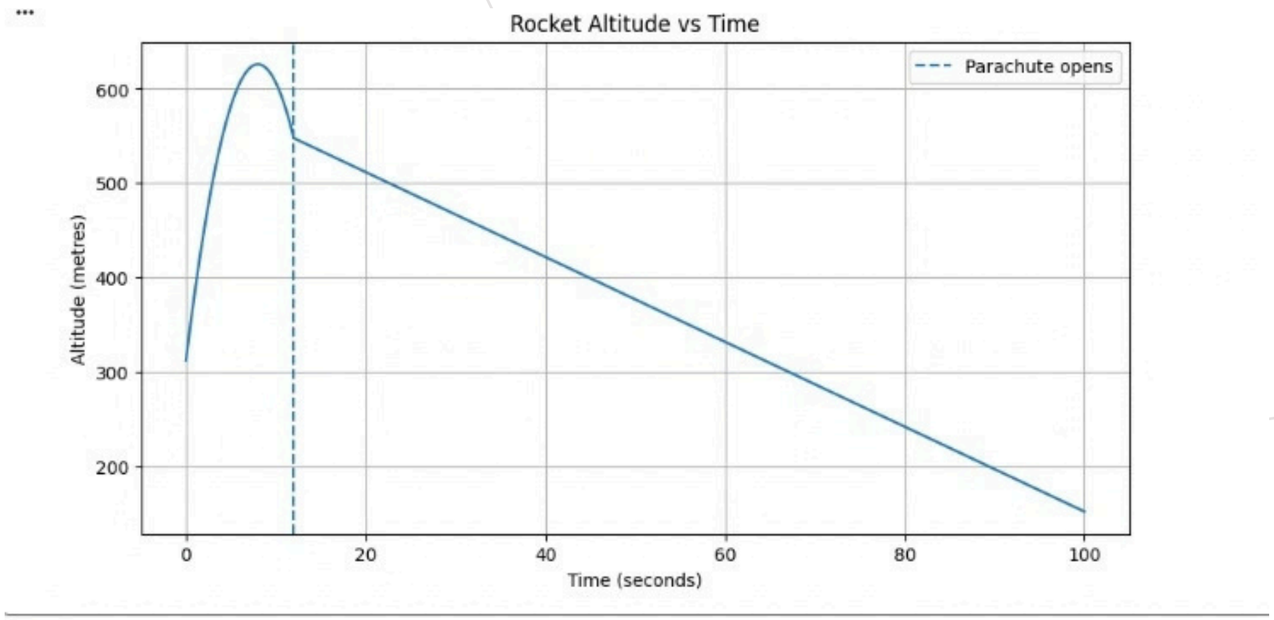


ROCKET FLIGHT MODEL

Two Distinct Mathematical Regimes

Physics of the Trajectory

- Phase 1: Boost & Coast (Parabolic)
- Governed by quadratic motion under gravity:
- $h(t) = h_0 + v_0 \cdot t - \frac{1}{2} \cdot g \cdot t^2$
- Phase 2: Parachute Descent (Terminal)
- Linear descent with constant terminal velocity:
- $h(t) = h_deploy + v_term \cdot (t - t_deploy)$
- Key Insight: Finding the intersection of these two models identifies parachute deployment.



BUILDING THE SOLUTION

Level 0: Baseline Verification

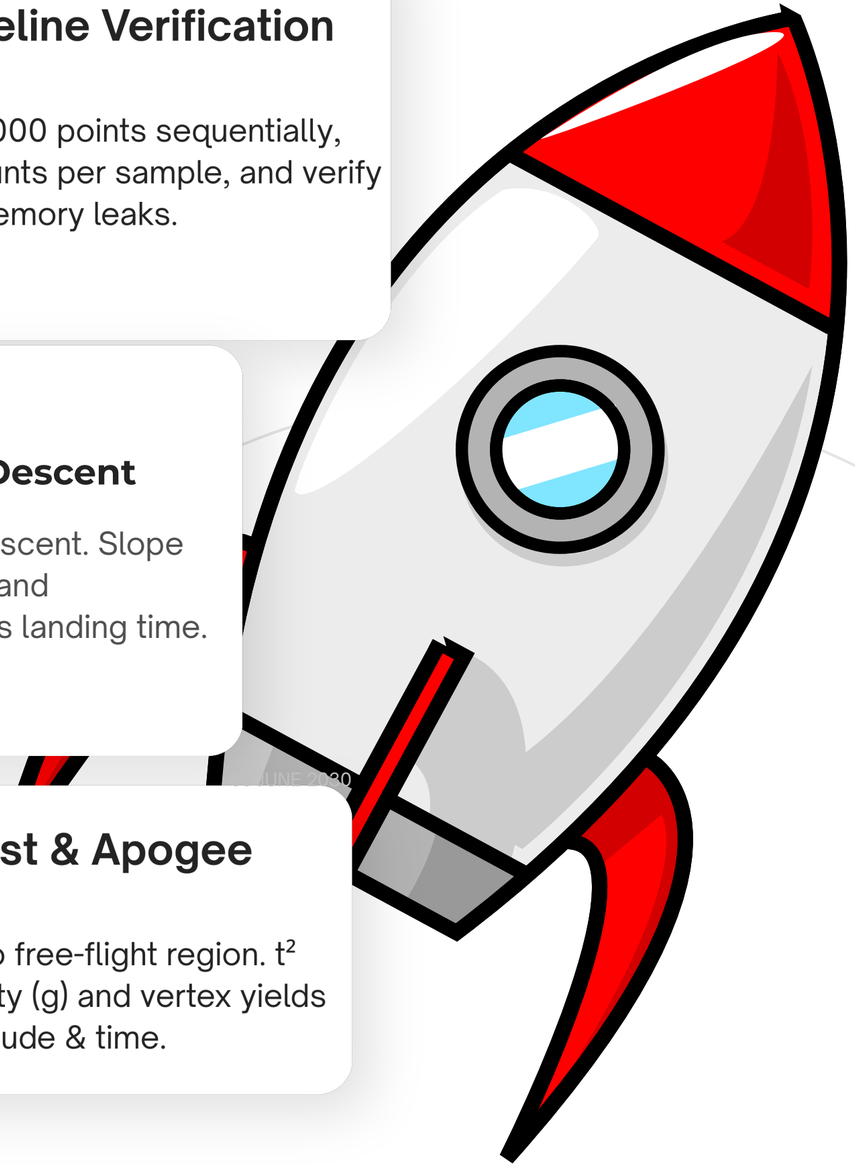
Read exactly 100,000 points sequentially, benchmark cycle counts per sample, and verify zero memory leaks.

Level 1: Parachute Descent

Isolate late-stage linear descent. Slope yields descent velocity (v) and extrapolation to h = 0 yields landing time.

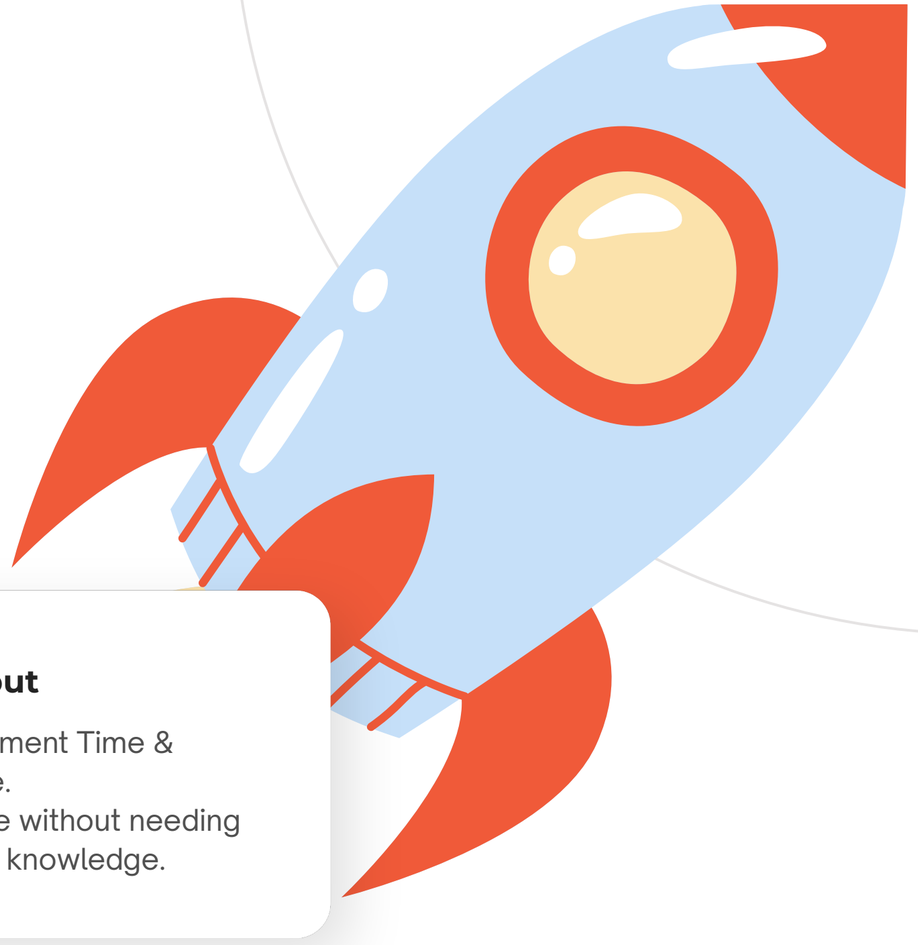
Level 2: Coast & Apogee

Fit quadratic curve to free-flight region. t^2 coefficient yields gravity (g) and vertex yields apogee altitude & time.



PARACHUTE DEPLOYMENT

Detecting the Quadratic-to-Linear Inflection Point



Backward Detection Algorithm

- Lock onto steady linear parachute descent.
- Scan backward until points deviate from the line.
- Fit quadratic curve to the confirmed coast data.
- Solve quadratic vs. line equation intersection.

Intersection Output

- Yields exact Deployment Time & Deployment Altitude.
- Robust against noise without needing prior flight schedule knowledge.

MEMORY OPTIMIZATION

Fitting 100,000 Samples into a 4,096-Byte Footprint

The Naive Way

100,000 × 4-byte floats = ~400 KB RAM
Instant Stack Overflow / 1OOM on ESP32

Block Statistics Approach

- 100 blocks × 1,000 samples
- Store only 3 running accumulators per block:
- Σh, $\Sigma(t \cdot h)$, $\Sigma(t^2 \cdot h)$
- Total Static RAM Used: 2,624 bytes
- Remaining Headroom: 1,472 bytes (36% buffer)



GLITCH HANDLING+



Outlier Rejection on Live Sensor Telemetry

Sensor Characteristics

Normal Noise: ±2 m variance
Sensor Glitches: ⩾50 m instantaneous spikes

Rejection Logic & Results

|h_current − h_last_good| > Threshold
If spike detected: discard & replace with last valid state.
Glitches Rejected: 739 samples (0.739%)

PERFORMANCE & CYCLES

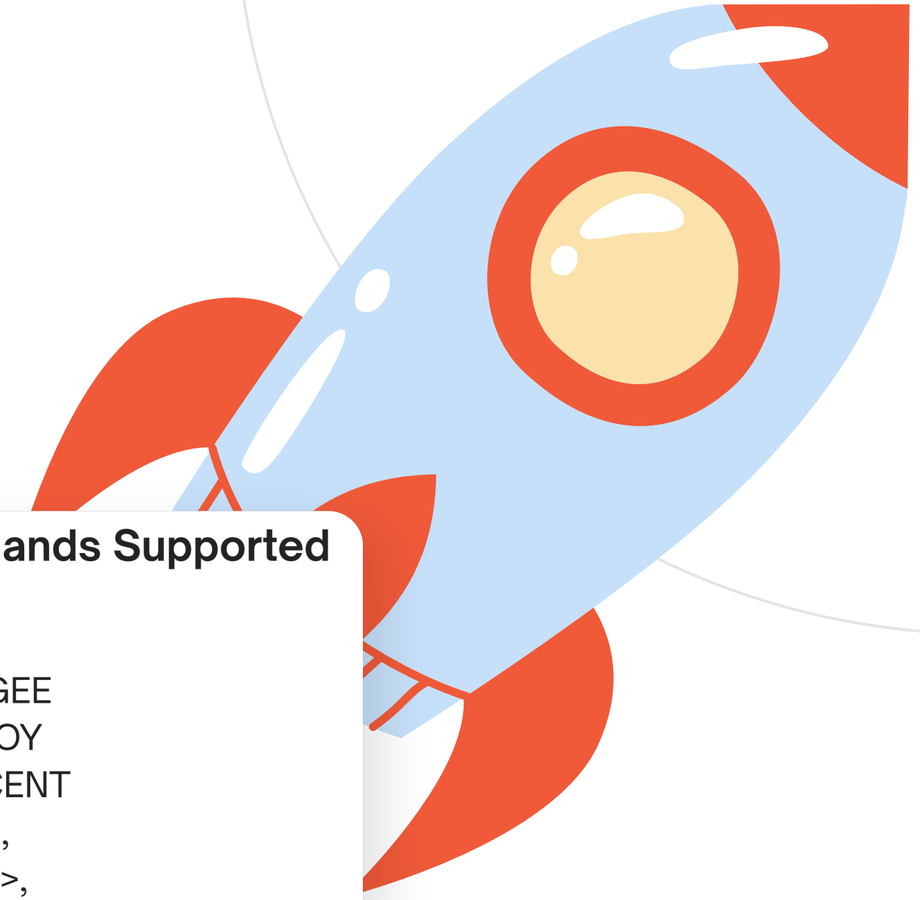
Real-Time Latency Verification on ESP32

System Benchmarks

- Memory: 2,672 / 4,096 bytes (35% safety headroom)
- Worst Reading Latency: 3,818 cycles / 12,000 budget
- CPU Utilization: Uses only 31.8% of allowed time!
- Readings Processed: Exactly 100,000 / 100,000.

+FLIGHT METRICS & COMMANDS

Official Tracker Verified Output



Computed Flight Values

- G = 9.811 m/s²
- APOGEE = 8.002 s (626.08 m)
- DEPLOY = 12.000 s (547.68 m)
  DESCENT = -4.500 m/s
- LAND = 133.70 s
- GLITCHES = 739

Commands Supported

- G
- APOGEE
- DEPLOY
- DESCENT
- LAND,
- ALT <t>,
- GLITCHES, STATS

LLM USAGE & TEAM COLLABORATION

What Worked, What Didn't, and Team Vitals

LLM Log :

- **Where it helped**: Centered polynomial regression math & Gaussian solver.
- **Where it was unhelpful/wrong: I**nitially suggested using data from a block containing both coast and descent data.
- This gave wrong results when deployment happened in the middle of a block.
- We changed it to first separate the coast and descent data, then fit each phase separately.

Team Vitals:

- **Hostel Sprint:** Sourabha & Pranusha worked in-person side-by-side.
- **Sync:** 1 All-hands Google Meet + 5 phone calls between Sourabha and Pranusha.
- **Average Sleep:** ~6.0 hrs/night

THANK YOU!



TEAM COMPUTEX